Ştefan Gheţu

Bucharest, Romania • +40 770 760 492 • stefanghetu9@gmail.com
github.com/ursus161

Education

University of Bucharest – Faculty of Mathematics and Computer Science *Bucharest*
BSc in Computer Science 2025-2028

- **Relevant coursework:** Computer Architecture, Linear & Abstract Algebra, Object-Oriented Programming, Data Structures & Algorithms, Formal Languages & Automata, Calculus, Databases.

NXP Linux Kernel Summer School *Bucharest*
Hands-on program on Linux kernel driver development: character devices, GPIOs and IRQs, and SPI/I2C peripheral drivers (ST7789 display, BMP280 sensor) on the NXP FRDM i.MX 93 board. 2026

- **Won 1st place** in the final hackathon with a retro system monitor: a **Linux kernel module** exposing CPU/memory spike detection fed by a **userspace daemon** parsing `/proc`. Built in **C** on the **FRDM i.MX 93**.

ARM Summer School *Bucharest*
ARM boot chain from scratch (TF-A, U-Boot), custom Linux with Buildroot, TrustZone/OP-TEE TEEs, on the ***NXP FRDM i.MX 93****.* 2026

- Afterwards, I starting looking into, alongside PhD graduates and students, and currently working on research for a **Energy-Aware CPU Hybrid Process Scheduler**.

NXP Cryptographic Algorithms in Embedded Systems Summer School *Bucharest*
Implemented and ran cryptographic algorithms in ***C*** *on the NXP LPC55S69 (****ARM Cortex-M33****) board, alongside sessions on embedded development and testing* 2026

Technical Skills

Languages	C, C++ (23), Python, x86-64 & RISC-V assembly, Bash, Rust, SQL, JavaScript
Systems & Arch	Pipelining, hazard detection and forwarding, cache hierarchies, low-latency, micro-benchmarking, concurrency
Embedded & Kernel	Linux kernel modules, character device drivers, GPIO/IRQ, SPI/I2C
Databases	**Oracle SQL**: schema design, normalization (BCNF/4NF/5NF), query optimization, views, TOP-N, DIVISION
Tools	Git, Linux (Ubuntu/Debian), GDB, ptrace, Make, Catch2, Docker, cron, minicom, LaTeX
Spoken	Romanian (native), English (C2)

Projects

RISC-V Pipeline Performance Analyzer `https://github.com/ursus161/riscv-performance-analyzer`
Python • FastAPI • Rust •React

- Cycle-accurate simulator in **Python**, currently rewriting the core in **Rust**, of a 5-stage RISC-V pipeline (IF/ID/EX/MEM/WB) with a two-pass RV32I assembler, full data forwarding, and hazard detection (RAW, load-use stalls); configurable **LRU/LFU** cache module.
- Implemented a **Perceptron branch predictor** (Jimenez & Lin threshold) with a global history register and collision-resistant hashing – an ML model applied to hardware; achieved 89.8% accuracy on synthetic benchmarks, reducing cycle count by ∼9% versus always-stall.
- Exposed the simulator as a **FastAPI** REST backend with stateful step-through sessions (UUID-managed), a `/compare` endpoint that benchmarks 30 cache configurations in parallel.

TaskSched – Real-Time Task Scheduler Simulator `https://github.com/ursus161/TaskSched`
C++23 *• OOP • Real-Time Systems • Discrete-Event Simulation*

- RTOS-inspired tick-based scheduler simulator in **C++23** built around a multi-class hierarchy, using virtual inheritance to model periodic, aperiodic, and sporadic task variants under a single `Task` abstraction.
- Implements multiple relevant RTOS scheduling policies behind a common `Scheduler` interface, driven by a top-level `Simulator` class.
- **UUniFast** task-set generation for schedulability experiments, soft real-time semantics with job dropping, and a CSV `TraceSink` pipeline for offline analysis of scheduling traces.
- In progress: multi-core model with per-core ready queues and inter-core load balancing for task migration, toward an adaptive partitioned scheduler with cache-aware migration penalties.

CortexPeek – x86-64 Userspace Debugger https://github.com/ursus161/cortexpeek
C++23 • ptrace • libcapstone • Catch2

- Built a **ptrace**-based debugger in **C++23** for ELF binaries: `INT3` breakpoints via masked `PEEKDATA`/`POKEDATA` read-modify-write, `PTRACE_SINGLESTEP`, GPR snapshots, and partial-word tracee memory I/O through a `TriviallyCopyable`-constrained `MemoryView<T>`.
- Solved the canonical INT3 resume: on `SIGTRAP`, rewind `RIP`, restore the original byte, single-step, reinject `0xCC` – the same hand-off GDB uses.
- Distinguished spawn vs. attach lifecycles (`fork`+`TRACEME`+`execvp` vs. `PTRACE_ATTACH`) with separate teardown paths to avoid zombie leaks; ELF symbol resolution via `nm` for name-based breakpoints and function-bounded disassembly via **libcapstone**.
- RAII over kernel handles, exclusive smart-pointer ownership, custom exception hierarchy, Command + Observer + Factory patterns, strictly verified by **Catch2** unit tests.

PackageMonitor https://github.com/ursus161/PackageMonitor
Bash • Python *• Cron • Linux*

- **Led a team** in developing a system-level tool that tracks `dpkg` package state on Debian/Ubuntu via scheduled **cron** jobs, with incremental processing and structured CSV output of system changes over time.

Experience & Activities

ACS Keysight Capture The Flag – 3rd Place *2026*
Keysight & ACS CTF; challenges spanning cryptography, reverse engineering, binary exploitation, forensics, and web
Bucharest

Co-Founder – Brăila Tech Sprint *2023-2025*
Co-organized the first two editions of Brăila's first hackathon (County Library partnership) Brăila